Name: Song Ting Yu 宋庭宇 Student ID: 314512091

1. The method I used is a method proposed by Di Zenzo which is an edge detector that works in vector space. The procedure is:
   1) We first use Sobel operator to get the Gradient at X-direction and Y-direction for each channel of the image.
   2) We then use the equation:

$$g_{xx} = \left|\frac{\partial R}{\partial x}\right|^2 + \left|\frac{\partial G}{\partial x}\right|^2 + \left|\frac{\partial B}{\partial x}\right|^2$$

$$g_{xx} = \left|\frac{\partial R}{\partial y}\right|^2 + \left|\frac{\partial G}{\partial y}\right|^2 + \left|\frac{\partial B}{\partial y}\right|^2$$

$$g_{xy} = \frac{\partial R}{\partial x}\frac{\partial R}{\partial y} + \frac{\partial G}{\partial x}\frac{\partial G}{\partial y} + \frac{\partial B}{\partial x}\frac{\partial B}{\partial y}$$

   3) After getting g_{xx}, g_{yy}, g_{xy}, we calculate the angle of maximum rate of change $\theta(x, y)$:

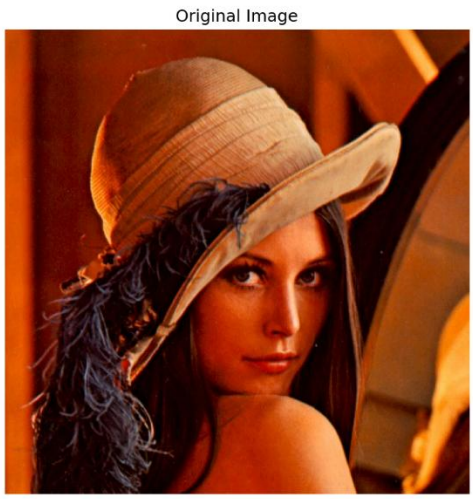
$$\theta(x, y) = \frac{1}{2}\tan^{-1}\left[\frac{2g_{xy}}{g_{xx} - g_{yy}}\right]$$

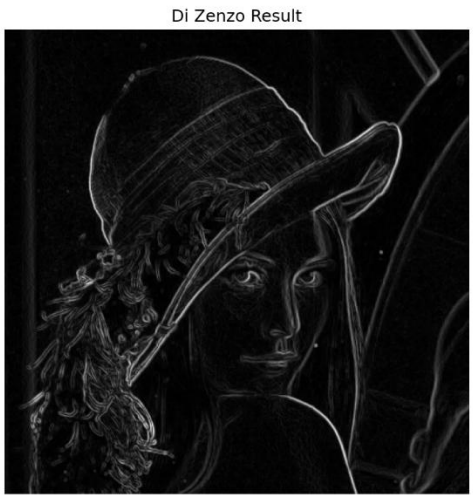
   4) And the value of rate of change is:

$$F_\theta(x, y) = \left\{\frac{1}{2}\left[\left(g_{xx} + g_{yy}\right) + \left(g_{xx} - g_{yy}\right)\cos 2\theta(x, y) + 2g_{xy} sin 2\theta(x, y)\right]\right\}^{\frac{1}{2}}$$

By going through the procedure, we would get the edge of the images using Di Zenzo method.

Results:





Source Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy import ndimage

def sobel_derivatives(channel):
    sobel_x = np.array([[-1, 0, 1],
                        [-2, 0, 2],
                        [-1, 0, 1]], dtype=np.float32)

    sobel_y = np.array([[-1, -2, -1],
                        [ 0,  0,  0],
                        [ 1,  2,  1]], dtype=np.float32)
    gx = ndimage.convolve(channel, sobel_x)
    gy = ndimage.convolve(channel, sobel_y)

    return gx, gy

def dizenzo_gradient(rgb_image):
    img = rgb_image.astype(np.float32)
    R = img[:, :, 0]
    G = img[:, :, 1]
    B = img[:, :, 2]
    Rx, Ry = sobel_derivatives(R)
    Gx, Gy = sobel_derivatives(G)
    Bx, By = sobel_derivatives(B)
    gxx = Rx**2 + Gx**2 + Bx**2
    gyy = Ry**2 + Gy**2 + By**2
    gxy = Rx*Ry + Gx*Gy + Bx*By
    theta = 0.5 * np.arctan2(2*gxy, gxx - gyy)
    term1 = gxx + gyy
    term2 = (gxx - gyy) * np.cos(2 * theta)
    term3 = 2 * gxy * np.sin(2 * theta)

    gradient_magnitude = np.sqrt(0.5 * (term1 + term2 + term3))

    gradient_direction = theta

    return gradient_magnitude, gradient_direction

def main():
    image_path = 'lenna-RGB.tif'

    try:
        img = cv2.imread(image_path, cv2.IMREAD_COLOR)

        if img is None:
```

```
            print(f"Error: Could not read image '{image_path}'")
            print("Please ensure the image file exists in the current
directory.")
            return
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        print(f"Image loaded successfully: {img_rgb.shape}")
        print(f"Image size: {img_rgb.shape[1]} x {img_rgb.shape[0]}")
        print(f"Color channels: {img_rgb.shape[2]}")
        print("\nApplying Di Zenzo gradient method...")
        gradient_mag, gradient_dir = dizenzo_gradient(img_rgb)
        gradient_normalized = cv2.normalize(gradient_mag, None, 0, 255,
cv2.NORM_MINMAX)
        gradient_normalized = gradient_normalized.astype(np.uint8)
        print(f"\nGradient Statistics:")
        print(f"Min gradient magnitude: {gradient_mag.min():.2f}")
        print(f"Max gradient magnitude: {gradient_mag.max():.2f}")
        print(f"Mean gradient magnitude: {gradient_mag.mean():.2f}")
        print(f"Std gradient magnitude: {gradient_mag.std():.2f}")
        fig, axes = plt.subplots(1, 2, figsize=(10, 5))
        axes[0].imshow(img_rgb)
        axes[0].set_title('Original Image', fontsize=12)
        axes[0].axis('off')
        axes[1].imshow(gradient_normalized, cmap='gray')
        axes[1].set_title('Di Zenzo Result', fontsize=12)
        axes[1].axis('off')

        plt.tight_layout()
        output_filename = 'dizenzo_gradient_result.png'
        plt.savefig(output_filename, dpi=150, bbox_inches='tight')
        print(f"\nResult saved as '{output_filename}'")
        cv2.imwrite('dizenzo_gradient_magnitude.png',
gradient_normalized)
        print(f"Gradient magnitude saved as
'dizenzo_gradient_magnitude.png'")

        plt.show()

        print("\n" + "="*60)
        print("PROCEDURE SUMMARY:")
        print("="*60)
        print("1. Load RGB color image 'lenna-RGB.tif'")
        print("2. Extract R, G, B channels")
        print("3. Apply Sobel operators to compute partial
derivatives:")
        print("   - Rx, Ry (Red channel derivatives)")
        print("   - Gx, Gy (Green channel derivatives)")
        print("   - Bx, By (Blue channel derivatives)")
```

```
        print("4. Compute metric tensor elements:")
        print("   - gxx = Rx² + Gx² + Bx²")
        print("   - gyy = Ry² + Gy² + By²")
        print("   - gxy = Rx·Ry + Gx·Gy + Bx·By")
        print("5. Calculate gradient direction using Eq. (6-55):")
        print("   - θ(x,y) = (1/2)·tan⁻¹[2·g_xy / (g_xx - g_yy)]")
        print("6. Calculate gradient magnitude using Eq. (6-56):")
        print("   - F_θ(x,y) = sqrt{(1/2)·[(g_xx + g_yy) +")
        print("                (g_xx - g_yy)·cos(2θ) +
2·g_xy·sin(2θ)]}")
        print("7. Normalize and display results")
        print("="*60)

    except Exception as e:
        print(f"An error occurred: {e}")
        import traceback
        traceback.print_exc()

if __name__ == "__main__":
    main()
```

2. Similar to the method used in question 1, the method I used is a method proposed by Di Zenzo which is an edge detector that works in vector space. The procedure is:
   1) We first use Sobel operator to get the Gradient at X-direction and Y-direction for each channel of the image.
   2) We then use the equation:

$$g_{xx} = \left|\frac{\partial R}{\partial x}\right|^2 + \left|\frac{\partial G}{\partial x}\right|^2 + \left|\frac{\partial B}{\partial x}\right|^2$$

$$g_{xx} = \left|\frac{\partial R}{\partial y}\right|^2 + \left|\frac{\partial G}{\partial y}\right|^2 + \left|\frac{\partial B}{\partial y}\right|^2$$

$$g_{xy} = \frac{\partial R}{\partial x}\frac{\partial R}{\partial y} + \frac{\partial G}{\partial x}\frac{\partial G}{\partial y} + \frac{\partial B}{\partial x}\frac{\partial B}{\partial y}$$

   3) After getting g_{xx}, g_{yy}, g_{xy}, we calculate the angle of maximum rate of change $\theta(x, y)$:

$$\theta(x, y) = \frac{1}{2}\tan^{-1}\left[\frac{2g_{xy}}{g_{xx} - g_{yy}}\right]$$

   4) And the value of rate of change is:

$$F_\theta(x,y) = \left\{\frac{1}{2}\left[\left(g_{xx} + g_{yy}\right) + \left(g_{xx} - g_{yy}\right)\cos 2\theta(x,y) + 2g_{xy} sin2\theta(x,y)\right]\right\}^{\frac{1}{2}}$$

By going through the procedure, we would get the edge of the images using Di Zenzo method.

Result:

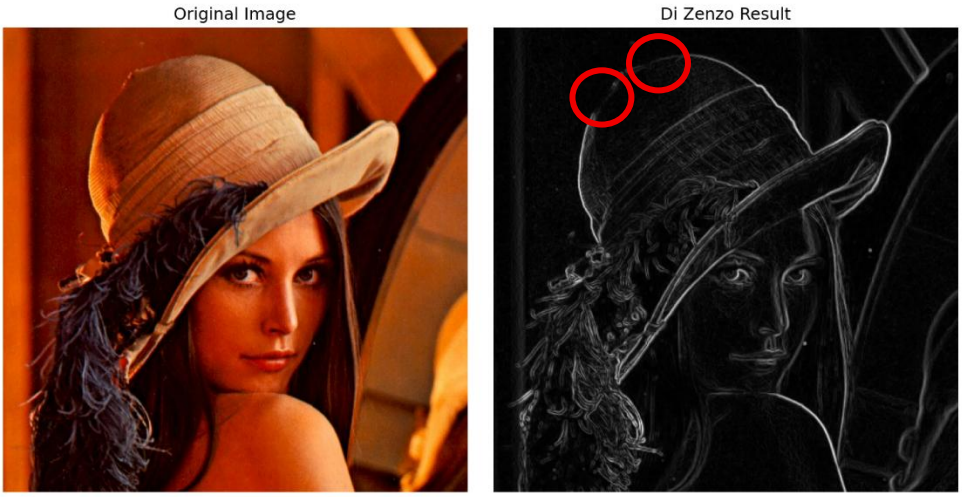
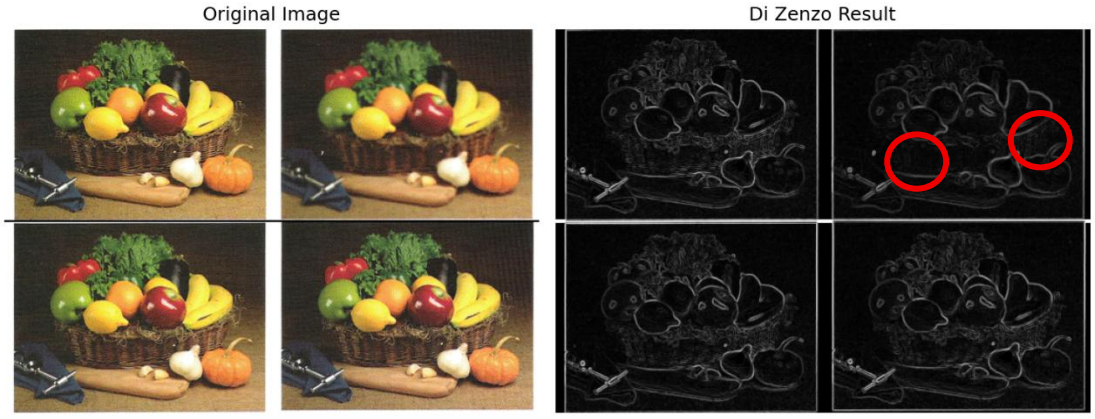
Source Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy import ndimage

def sobel_derivatives(channel):
    sobel_x = np.array([[-1, 0, 1],
                        [-2, 0, 2],
                        [-1, 0, 1]], dtype=np.float32)

    sobel_y = np.array([[-1, -2, -1],
                        [ 0,  0,  0],
                        [ 1,  2,  1]], dtype=np.float32)
    gx = ndimage.convolve(channel, sobel_x)
    gy = ndimage.convolve(channel, sobel_y)

    return gx, gy

def dizenzo_gradient(rgb_image):
    img = rgb_image.astype(np.float32)
    R = img[:, :, 0]
    G = img[:, :, 1]
    B = img[:, :, 2]
```

```
    Rx, Ry = sobel_derivatives(R)
    Gx, Gy = sobel_derivatives(G)
    Bx, By = sobel_derivatives(B)
    gxx = Rx**2 + Gx**2 + Bx**2
    gyy = Ry**2 + Gy**2 + By**2
    gxy = Rx*Ry + Gx*Gy + Bx*By
    theta = 0.5 * np.arctan2(2*gxy, gxx - gyy)
    term1 = gxx + gyy
    term2 = (gxx - gyy) * np.cos(2 * theta)
    term3 = 2 * gxy * np.sin(2 * theta)

    gradient_magnitude = np.sqrt(0.5 * (term1 + term2 + term3))

    gradient_direction = theta

    return gradient_magnitude, gradient_direction

def main():
    image_path = 'Visual resolution.gif'

    try:
        img = cv2.imread(image_path, cv2.IMREAD_COLOR)

        if img is None:
            print(f"Error: Could not read image '{image_path}'")
            print("Please ensure the image file exists in the current
directory.")
            return
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        print(f"Image loaded successfully: {img_rgb.shape}")
        print(f"Image size: {img_rgb.shape[1]} x {img_rgb.shape[0]}")
        print(f"Color channels: {img_rgb.shape[2]}")
        print("\nApplying Di Zenzo gradient method...")
        gradient_mag, gradient_dir = dizenzo_gradient(img_rgb)
        gradient_normalized = cv2.normalize(gradient_mag, None, 0, 255,
cv2.NORM_MINMAX)
        gradient_normalized = gradient_normalized.astype(np.uint8)
        print(f"\nGradient Statistics:")
        print(f"Min gradient magnitude: {gradient_mag.min():.2f}")
        print(f"Max gradient magnitude: {gradient_mag.max():.2f}")
        print(f"Mean gradient magnitude: {gradient_mag.mean():.2f}")
        print(f"Std gradient magnitude: {gradient_mag.std():.2f}")
        fig, axes = plt.subplots(1, 2, figsize=(10, 5))
        axes[0].imshow(img_rgb)
        axes[0].set_title('Original Image', fontsize=12)
        axes[0].axis('off')
        axes[1].imshow(gradient_normalized, cmap='gray')
```

```
        axes[1].set_title('Di Zenzo Result', fontsize=12)
        axes[1].axis('off')

        plt.tight_layout()
        output_filename = 'dizenzo_gradient_result.png'
        plt.savefig(output_filename, dpi=150, bbox_inches='tight')
        print(f"\nResult saved as '{output_filename}'")
        cv2.imwrite('dizenzo_gradient_magnitude.png',
gradient_normalized)
        print(f"Gradient magnitude saved as
'dizenzo_gradient_magnitude.png'")

        plt.show()

        print("\n" + "="*60)
        print("PROCEDURE SUMMARY:")
        print("="*60)
        print("1. Load RGB color image 'lenna-RGB.tif'")
        print("2. Extract R, G, B channels")
        print("3. Apply Sobel operators to compute partial
derivatives:")
        print("   - Rx, Ry (Red channel derivatives)")
        print("   - Gx, Gy (Green channel derivatives)")
        print("   - Bx, By (Blue channel derivatives)")
        print("4. Compute metric tensor elements:")
        print("   - gxx = Rx² + Gx² + Bx²")
        print("   - gyy = Ry² + Gy² + By²")
        print("   - gxy = Rx·Ry + Gx·Gy + Bx·By")
        print("5. Calculate gradient direction using Eq. (6-55):")
        print("   - θ(x,y) = (1/2)·tan⁻¹[2·g_xy / (g_xx - g_yy)]")
        print("6. Calculate gradient magnitude using Eq. (6-56):")
        print("   - F_θ(x,y) = sqrt{(1/2)·[(g_xx + g_yy) +")
        print("                (g_xx - g_yy)·cos(2θ) +
2·g_xy·sin(2θ)]}")
        print("7. Normalize and display results")
        print("="*60)

    except Exception as e:
        print(f"An error occurred: {e}")
        import traceback
        traceback.print_exc()

if __name__ == "__main__":
    main()
```

3. So, the 2 designs I used are similar designs which are inspired by Di Zenzo's method. We can see the method yields rather good result for both "lenna-RGB.tif" & "Visual resolution.gif". For the lenna-RGC.tif we can see that some edges are missing due to the edges being similar to their surroundings:



Same goes to the Visual resolution.gif the basket at the upper right image does not seem visible due to it being blurry and similar in color with its surroundings so its gradient is very small so there are less visible edges there: